Though it has many advantages over Excel, SQL is still used only for simple tasks like aggregating data, joining data sets, and performing complex statistical and mathematical computations.

SQL is also rich in data analysis features. A discussion covering all of them in this limited space is not feasible. In this article, I will explore some of the most useful data analysis features of SQL. Its other important features and its data analysis functionalities will be discussed later.

Data processing

Typically, data processing means operating on numerical and character strings. SQL supports both and provides various operators for them. Among different arithmetic, logical and relational operators, IS NUL, BETWEEN, IN and EXISTS are the most important and provide powerful searching options that are frequently used for data analysis tasks.

Resources

The analysis has been done on Northwind and *world.sql* RDBMS. These are available in the Google archives *https://code.google.com/archive/p/northwindextended/downloads* and *https://dev.mysql.com/doc/index-other.html*. Readers can download the data and experiment with it in their own way.

Data analysis

Since SQL is advantageous for storage and security reasons, it is widely used for storage, the aggregation of data, joining of tables, etc. Here I will discuss a few data analysis features of SQL.

Grouping

Aggregation of field values on different categories of a given field is a good option for category-wise data analysis and visualisation. For example, in a world database, to add the population of all cities of a country, we can group the city table on the basis of the country's identification code.

```
use world
select a.CountryCode, sum(a.
Population)
from city as a
group by a.CountryCode
order by a.CountryCode
limit 5
```

This grouping can be made selective by using the HAVING clause. For instance, if the data analysis requires the total population figures of a few selected countries only, we can write the code using IN and BETWEEN as follows:

```
select a.CountryCode, sum(a.
Population) as Population
from city as a
group by a.CountryCode
having a.CountryCode in
("ABW","AFG","ALB")
order by a.CountryCode
limit 5
```

The analysis report of the above query is as follows:

CountryCode	Population
ABW	29034
AFG	2332100
ALB	270000

```
select a.CountryCode, sum(a.
Population) as Population
from city as a
group by a.CountryCode
having a.CountryCode between "ABW" and
"ALB"
order by a.CountryCode
limit 5
```

The analysis report is as follows:

CountryCode	Population
ABW	29034
AFG	2332100
AGO	2561600
AIA	1556
ALB	270000

Joining tables

Joining multiple tables using their Cartesian product is an important feature of SQL. It provides the true benefit of a relational database system. Combined virtual table, of related data, mapped through the key fields, is frequently used in data analysis. For example, to replace the country code with the actual name we can join the city table with the country table on the basis of the country code, and then replace it with the actual name in the report.

```
 select b.name,sum(a.Population) as
Population
from city as a
join country as b on b.Code=a.
CountryCode
group by a.CountryCode
order by b.name
```

The analysis report is as follows:

Name	Population
Afghanistan	2332100
Albania	270000
Algeria	5192179
American Samoa	7523
Andorra	21189

Self-join

Self-join is used to analyse a table with respect to itself. It has several utilities. Here I have used this concept to identify those employees who have